```java
package com.mycompany.practicaexamen;

import java.rmi.Remote;
import java.rmi.RemoteException;

/**
 *
 * @author Jhendef
 */
public interface IJusticia extends Remote{
    public RespuestaCuenta ConsultarCuentas(String ci, String nombres, String apellidos) throws RemoteException;
    public Boolean Congelar(Cuenta cuenta, Double monto) throws RemoteException;
}
```

```java
package com.mycompany.practicaexamen;

import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.net.Socket;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import java.util.ArrayList;

/**...4 lines */
public class Justicia extends UnicastRemoteObject implements IJusticia {

    public Justicia() throws RemoteException {
        super();
    }

    @Override
    public RespuestaCuenta ConsultarCuentas(String ci, String nombres, String apellidos) throws RemoteException {
        RespuestaCuenta respuesta = new RespuestaCuenta(false, "Consulta realizada", new ArrayList<Cuenta>());
        try (Socket tcpsocket = new Socket("localhost", 5001);
                DataInputStream input = new DataInputStream(tcpsocket.getInputStream());
                DataOutputStream output = new DataOutputStream(tcpsocket.getOutputStream()))//Conexion con el TCP y declaracion de las variables de envio y recepcion
        {
            output.writeUTF(ci); //Envia a TCP
            output.writeUTF("buscar");
            String respuestaTCP = input.readUTF(); //Recibe la respuesta de TCP
            procesar(respuestaTCP, Bancos.MERCANTIL, ci, nombres, apellidos, respuesta);

        } catch (IOException e) {
            System.out.println("Error con Mercantil: " + e.getMessage());
        }
        try (DatagramSocket udpSocket = new DatagramSocket())//Conexion a UDP
```

```java
        try (DatagramSocket udpSocket = new DatagramSocket())//Conexion a UDP
        {
            String mensaje = "buscar:" + ci;
            byte[] buffer = mensaje.getBytes();
            DatagramPacket paquete = new DatagramPacket(buffer, buffer.length, InetAddress.getByName("localhost"), 5002);
            udpSocket.send(paquete);//Envia el paquete

            byte[] recibirBuffer = new byte[1024];
            DatagramPacket recibido = new DatagramPacket(recibirBuffer, recibirBuffer.length);
            udpSocket.receive(recibido);//recibe la respuesta

            String respuestaUDP = new String(recibido.getData(), 0, recibido.getLength());
            procesar(respuestaUDP, Bancos.BCP, ci, nombres, apellidos, respuesta);

        } catch (IOException e) {
            System.out.println("Error con BCP: " + e.getMessage());
        }
        return respuesta;
    }

    @Override
    public Boolean Congelar(Cuenta cuenta, Double monto) throws RemoteException {
        boolean exito = false;
        if (cuenta.getBanco() == Bancos.MERCANTIL) {
            try (Socket tcpsocket = new Socket("localhost", 5001); DataInputStream in = new DataInputStream(tcpsocket.getInputStream()); DataOutputStream out = new DataOutputStrea
                out.writeUTF(cuenta.getCi());
                out.writeUTF("congelar");
                out.writeUTF(cuenta.getNrocuenta());
                out.writeDouble(monto);
                exito = in.readBoolean();
            } catch (IOException e) {
                System.out.println("Error TCP al congelar: " + e.getMessage());
            }
        } else if (cuenta.getBanco() == Bancos.BCP) {
```

```java
        } else if (cuenta.getBanco() == Bancos.BCP) {
            try (DatagramSocket udpSocket = new DatagramSocket()) {
                String msg = "congelar:" + cuenta.getNrocuenta() + ":" + monto;
                byte[] buffer = msg.getBytes();
                DatagramPacket pqtRequest = new DatagramPacket(buffer, buffer.length, InetAddress.getByName("localhost"), 5002);
                udpSocket.send(pqtRequest);
                byte[] receiveBuffer = new byte[1024];
                DatagramPacket pqtResponse = new DatagramPacket(receiveBuffer, receiveBuffer.length);
                udpSocket.receive(pqtResponse);
                String resUDP = new String(pqtResponse.getData(), 0, pqtResponse.getLength());
                exito = resUDP.trim().equalsIgnoreCase("SI");
            } catch (IOException e) {
                System.out.println("Error UDP al congelar: " + e.getMessage());
            }
        }
        return exito;
    }

    private void procesar(String respuestas, Bancos banco, String ci, String nombres, String apellidos, RespuestaCuenta respuesta) {
        if (respuestas == null) {
            return;
        }
        String[] registros = respuestas.split(":");
        for (String reg : registros) {
            String[] datos = reg.split("-");
            respuesta.cuentas.add(new Cuenta(banco, datos[0], ci, nombres, apellidos, Double.parseDouble(datos[1])));
        }
    }
}
```

```java
package com.mycompany.practicaexamen;

import java.net.MalformedURLException;
import java.rmi.AlreadyBoundException;
import java.rmi.Naming;
import java.rmi.RemoteException;
import java.rmi.registry.LocateRegistry;

/**
 *
 * @author Jhendef
 */
public class ServidorJusticia {

    public static void main(String[] args) throws AlreadyBoundException {
        try {
            Justicia justicia = new Justicia();
            LocateRegistry.createRegistry(1099);
            Naming.bind("nameJusticia", justicia);
        } catch (RemoteException ex) {
            System.getLogger(ServidorJusticia.class.getName()).log(System.Logger.Level.ERROR, (String) null, ex);
        } catch (AlreadyBoundException ex) {
            System.getLogger(ServidorJusticia.class.getName()).log(System.Logger.Level.ERROR, (String) null, ex);
        } catch (MalformedURLException ex) {
            System.getLogger(ServidorJusticia.class.getName()).log(System.Logger.Level.ERROR, (String) null, ex);
        }
    }
}
```

```java
package com.mycompany.practicaexamen;

import java.net.MalformedURLException;
import java.rmi.Naming;
import java.rmi.NotBoundException;
import java.rmi.RemoteException;
import java.util.Scanner;

/**...4 lines */
public class Juez {

    public static void main(String[] args) throws NotBoundException, MalformedURLException, RemoteException {
        try {
            IJusticia cn = (IJusticia) Naming.lookup("rmi://localhost/nameJusticia"); //Conexion con el rmi
            Scanner sc = new Scanner(System.in);
            int op;
            do {
                System.out.println("Operaciones");
                System.out.println("1.Solicitar informacion");
                System.out.println("2.Ordenar congelamiento cuentas");
                System.out.println("3.Salir");
                op = sc.nextInt(); //Capturar enteros
                sc.nextLine();
                switch (op) {
                    case 1:
                        System.out.println("Ingresa el C.I.-");
                        String ci = sc.nextLine(); //Capturar String
                        System.out.println("Nombres:");
                        String nombres = sc.nextLine();
                        System.out.println("Apellidos:");
                        String apellidos = sc.nextLine();
                        RespuestaCuenta res = cn.ConsultarCuentas(ci, nombres, apellidos);
                        for (Cuenta c : res.cuentas) {
                            System.out.println(c.getBanco() + " - Cuenta: " + c.getNrocuenta() + " - Saldo: " + c.getSaldo());
                        }
                        break;
                    case 2:
```

```java
                    case 2:
                        System.out.print("Ingrese el CI para buscar cuentas a congelar: ");
                        String ciBusca = sc.nextLine();
                        RespuestaCuenta respuesta = cn.ConsultarCuentas(ciBusca, "Juan", "Perez");

                        if (respuesta.cuentas.isEmpty()) {
                            System.out.println("No se encontraron cuentas");
                        } else {
                            for (int i = 0; i < respuesta.cuentas.size(); i++) {
                                System.out.println(i + ". " + respuesta.cuentas.get(i).getNrocuenta() + " (" + respuesta.cuentas.get(i).getBanco() + ")");
                            }
                            System.out.print("Seleccione el indice de la cuenta: ");
                            int idx = sc.nextInt();
                            System.out.print("Ingrese el monto a congelar: ");
                            Double montoC = sc.nextDouble();

                            if (cn.Congelar(respuesta.cuentas.get(idx), montoC)) {
                                System.out.println("Fondos congelados");
                            } else {
                                System.out.println("Error al congelar");
                            }
                        }
                        break;
                    default:
                        break;
                }
            } while (op != 3);
        }
        catch (RemoteException | NotBoundException | MalformedURLException ex) {
            System.out.println("Error: " + ex.getMessage());
        }
    }
}
```

```java
package com.mycompany.practicaexamen;

import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

/**...4 lines */
public class ServerMercantil {
    public static void main(String[] args) throws IOException {
        ServerSocket server = new ServerSocket(5001);
        System.out.println("MercantilTCP listo");
        while(true){
            try(Socket cliente = server.accept();
                    DataInputStream input = new DataInputStream(cliente.getInputStream());
                    DataOutputStream output = new DataOutputStream(cliente.getOutputStream())
                    ){
                String ci = input.readUTF(); //Recive los datos enviados por el cliente
                String op = input.readUTF();
                if(op.equals("buscar")) //Manera correcta de comparar cadenas en java
                {
                    String respuesta = ci.equals("11021654") ? "1515-5200" : "";
                    output.writeUTF(respuesta); //Enviar la respuesta del TCP
                }else if(op.equals("congelar")){
                    String nroCuenta = input.readUTF();
                    Double monto = input.readDouble();
                    System.out.println("Congelando " + monto + " en cuenta " + nroCuenta);
                    output.writeBoolean(true);
                }
            }
        }
    }
}
```

```java
package com.mycompany.practicaexamen;

import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.SocketException;

/**...4 lines */
public class ServidorBCP {
    public static void main(String[] args) throws SocketException, IOException {
        DatagramSocket socket = new DatagramSocket(5002);
        System.out.println("Banco BCPUDP listo");
        while(true){
            byte[] buffer = new byte[1024];
            DatagramPacket peticion = new DatagramPacket(buffer, buffer.length);
            socket.receive(peticion); //Se recive el paquete del cliente

            String data = new String(peticion.getData(), 0, peticion.getLength());//Se descodifica de bytes a cadena
            String[] partes = data.split(":"); //se corta la cadena por los :
            String op = partes[0];
            if (op.equals("buscar")) {
                String ci = partes[1];
                String respuesta = ci.equals("11021654") ? "657654-6000" : "";
                byte[] mensaje = respuesta.getBytes(); //Se codifica la respuesta
                socket.send(new DatagramPacket(mensaje, mensaje.length, peticion.getAddress(), peticion.getPort())); //Envia datos al cliente
            }else if(op.equals("congelar")){
                String nroCuenta = partes[1];
                String monto = partes[2];
                System.out.println("BCP congelando " + monto + " en cuenta " + nroCuenta);
                byte[] msg = "SI".getBytes();
                socket.send(new DatagramPacket(msg, msg.length, peticion.getAddress(), peticion.getPort()));
            }
        }
    }
}
```

```java
package com.mycompany.practicaexamen;

import java.io.Serializable;

/**...4 lines */
public class Cuenta implements Serializable {
    public Bancos banco;
    public String nrocuenta;
    public String ci;
    public String nombres;
    public String apellidos;
    public Double saldo;

    public Cuenta(Bancos banco, String nrocuenta, String ci, String nombres, String apellidos, Double saldo){
        this.banco = banco;
        this.nrocuenta = nrocuenta;
        this.ci = ci;
        this.nombres = nombres;
        this.apellidos = apellidos;
        this.saldo = saldo;
    }

    public Bancos getBanco() {
        return banco;
    }

    public void setBanco(Bancos banco) {
        this.banco = banco;
    }
```

```java
package com.mycompany.practicaexamen;

/**...4 lines */
public enum Bancos {
    MERCANTIL, BCP;
}

```

```java
package com.mycompany.practicaexamen;

import java.io.Serializable;
import java.util.ArrayList;

/**...4 lines */
public class RespuestaCuenta implements Serializable{
    public Boolean error;
    public String mensaje;
    public ArrayList<Cuenta> cuentas;

    public RespuestaCuenta(Boolean error, String mensaje, ArrayList cuentas){
        this.error = error;
        this.mensaje = mensaje;
        this.cuentas = cuentas;
    }

}
```

Holas mundos

```java
public class ServerTCP {
    public static void main(String[] args) {
        int port = 5001;

        // El ServerSocket se crea fuera del while para evitar el error
        try (ServerSocket server = new ServerSocket(port)) {
            System.out.println("Servidor iniciado en el puerto " + port);
            while (true) {
                System.out.println("Esperando a un cliente...");
                try (Socket client = server.accept(); // Espera y acepta la conexion de un cliente
                     BufferedReader fromClient = new BufferedReader(new InputStreamReader(client.getInputStream()));//Lector de datos del cliente
                     PrintWriter toClient = new PrintWriter(client.getOutputStream(), true); // Escritor de datos hacia el cliente
                     Scanner sc = new Scanner(System.in)) { // Escaner para escribir desde el teclado del servidor
                    System.out.println("Cliente conectado desde " + client.getInetAddress() + ".");
                    String msgRecibido;
                    // Bucle de conversación: lee mientras el cliente no envie "salir"
                    while ((msgRecibido = fromClient.readLine()) != null) {
                        System.out.println("Cliente dice: " + msgRecibido);

                        if (msgRecibido.equalsIgnoreCase("salir")) break;

                        System.out.print("Tu respuesta (Servidor): ");
                        String respuesta = sc.nextLine();
                        toClient.println(respuesta); // Envia la respuesta al cliente
                    }
                } catch (IOException ex) {
                    System.out.println("Error con el cliente: " + ex.getMessage());
                }
            }
        } catch (IOException ex) {
            System.out.println("No se pudo iniciar el servidor: " + ex.getMessage());
        }
    }
}
```

```java
public class ClienteTCP {
    public static void main(String[] args) {
        int port = 5001;
        String host = "localhost";

        try (Socket socket = new Socket(host, port); // Abre la conexion con el servidor
             PrintWriter toServer = new PrintWriter(socket.getOutputStream(), true); // Flujo para enviar datos al servidor
             BufferedReader fromServer = new BufferedReader(new InputStreamReader(socket.getInputStream())); // Flujo para recibir datos del serv
             Scanner sc = new Scanner(System.in)) { // Escaner para entrada de teclado

            System.out.println("Conectado al servidor. Escribe 'salir' para terminar.");
            String mensaje;

            while (true) {
                System.out.print("Tu (Cliente): ");
                mensaje = sc.nextLine();
                toServer.println(mensaje); // Envia el mensaje al servidor

                if (mensaje.equalsIgnoreCase("salir")) break;

                String respuesta = fromServer.readLine(); // Recibe la respuesta del servidor
                System.out.println("Servidor responde: " + respuesta);
            }
        } catch (IOException ex) {
            System.out.println("Error de conexion: " + ex.getMessage());
        }
    }
}
```

```java
17  public class ServerMultiTCP {
18      public static void main(String[] args) throws IOException {
19          // El servidor escucha en el puerto 5056
20          ServerSocket ss = new ServerSocket(5056);
21          System.out.println("--- Servidor de Chat Iniciado ---");
22
23          while (true) {
24              Socket s = null;
25              try {
26                  // Bloqueado hasta que llegue un cliente
27                  s = ss.accept();
28                  System.out.println("Nuevo cliente conectado: " + s);
29
30                  // Canales de comunicación
31                  DataInputStream dis = new DataInputStream(s.getInputStream());
32                  DataOutputStream dos = new DataOutputStream(s.getOutputStream());
33
34                  // Crear el gestor (Handler) y arrancar el hilo
35                  Thread t = new ClienteHandler(s, dis, dos);
36                  t.start();
37
38              } catch (Exception e) {
39                  if (s != null) s.close();
40                  e.printStackTrace();
41              }
42          }
43      }
44  }
```

```java
16  class ClienteHandler extends Thread {
17      final DataInputStream dis;
18      final DataOutputStream dos;
19      final Socket s;
20      public ClienteHandler(Socket s, DataInputStream dis, DataOutputStream dos) {
21          this.s = s;
22          this.dis = dis;
23          this.dos = dos;
24      }
25      @Override
26      public void run() {
27          String recibido;
28          while (true) {
29              try {
30                  // El servidor espera el mensaje del cliente
31                  recibido = dis.readUTF();
32                  if (recibido.equalsIgnoreCase("Exit")) {
33                      System.out.println("El cliente " + this.s + " se ha desconectado.");
34                      this.dos.writeUTF("Desconexion exitosa");
35                      break;
36                  }
37                  // Log en la consola del servidor
38                  System.out.println("Cliente dice: " + recibido);
39                  // El servidor responde al cliente
40                  String respuesta = "Servidor recibio: [" + recibido + "]";
41                  dos.writeUTF(respuesta);
42              } catch (IOException e) {
43                  System.out.println("Conexion perdida con el cliente.");
44                  break;
45              }
46          }
47          // Cierre de flujos
```

```java
47          // Cierre de flujos
48          try {
49              this.dis.close();
50              this.dos.close();
51              this.s.close();
52          } catch (IOException e) {
53              e.printStackTrace();
54          }
55      }
56  }
```

```java
public class ClienteMultiTCP {
    public static void main(String[] args) {
        try (Scanner scn = new Scanner(System.in)) {
            // Usamos localhost para pruebas locales o la IP 10.8.221.221 si es en red
            InetAddress ip = InetAddress.getByName("localhost");

            try (Socket s = new Socket(ip, 5056);
                 DataInputStream dis = new DataInputStream(s.getInputStream());
                 DataOutputStream dos = new DataOutputStream(s.getOutputStream())) {

                System.out.println("Conectado al chat. Escribe tus mensajes (Escribe 'Exit' para salir):");

                while (true) {
                    System.out.print("> ");
                    String paraEnviar = scn.nextLine();
                    dos.writeUTF(paraEnviar);

                    // Si enviamos Exit, leemos la ultima respuesta y salimos
                    if (paraEnviar.equalsIgnoreCase("Exit")) {
                        System.out.println(dis.readUTF());
                        break;
                    }
                    // Leer lo que el servidor nos responde
                    String respuestaServidor = dis.readUTF();
                    System.out.println(respuestaServidor);
                }
            }
        } catch (IOException e) {
            System.out.println("No se pudo conectar con el servidor: " + e.getMessage());
        }
    }
}
```

```java
public class ServidorUDP {
    public static void main(String[] args) {
        try {
            // Creamos el socket en el puerto 5057
            DatagramSocket ds = new DatagramSocket(5057);
            byte[] bufferRecibo = new byte[1024];
            System.out.println("Servidor UDP esperando mensajes...");
            while (true) {
                // 1. Preparar el espacio para el paquete que viene
                DatagramPacket dpRecibo = new DatagramPacket(bufferRecibo, bufferRecibo.length);
                // 2. Recibir el paquete (el programa se detiene aquí hasta que llegue algo)
                ds.receive(dpRecibo);
                // 3. Convertir los bytes a texto
                String mensaje = new String(dpRecibo.getData(), 0, dpRecibo.getLength());
                System.out.println("Cliente dice: " + mensaje);
                // 4. (Opcional) Enviar una respuesta rápida
                String respuesta = "Mensaje recibido correctamente";
                byte[] bufferEnvio = respuesta.getBytes();
                DatagramPacket dpEnvio = new DatagramPacket(
                    bufferEnvio, bufferEnvio.length,
                    dpRecibo.getAddress(), dpRecibo.getPort()
                );
                ds.send(dpEnvio);
                // Si el mensaje es "exit", podrías cerrar el servidor si quisieras
                if (mensaje.equalsIgnoreCase("exit")) break;
            }
            ds.close();
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

```java
public class ClienteUDP {
    public static void main(String[] args) {
        try (Scanner sc = new Scanner(System.in)) {
            DatagramSocket ds = new DatagramSocket();
            InetAddress ip = InetAddress.getByName("localhost"); // O la IP del servidor
            System.out.println("--- Chat UDP Simple ---");
            while (true) {
                System.out.print("Escribe un mensaje: ");
                String mensaje = sc.nextLine();
                byte[] bufferEnvio = mensaje.getBytes();
                // 1. Crear el paquete con el mensaje, la IP y el puerto del servidor
                DatagramPacket dpEnvio = new DatagramPacket(bufferEnvio, bufferEnvio.length, ip, 5057);
                // 2. Enviar el paquete
                ds.send(dpEnvio);
                if (mensaje.equalsIgnoreCase("exit")) break;
                // 3. Esperar la respuesta del servidor
                byte[] bufferRecibo = new byte[1024];
                DatagramPacket dpRecibo = new DatagramPacket(bufferRecibo, bufferRecibo.length);
                ds.receive(dpRecibo);
                String respuesta = new String(dpRecibo.getData(), 0, dpRecibo.getLength());
                System.out.println("Servidor: " + respuesta);
            }
            ds.close();
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

```java
class Datos implements Serializable {
    String info;
    public Datos(String info) { this.info = info; }
}

public interface IHolaRMI extends Remote {
    // El cliente envía texto y un objeto, el servidor responde con un texto
    String enviarAlServidor(String texto, Datos objeto) throws RemoteException;

    // El servidor devuelve un objeto al cliente
    Datos recibirDelServidor() throws RemoteException;
}
```

```java
public class HolaRMI extends UnicastRemoteObject implements IHolaRMI {

    // Constructor obligatorio
    public HolaRMI() throws RemoteException {
        super();
    }

    @Override
    public String enviarAlServidor(String texto, Datos objeto) throws RemoteException {
        System.out.println("El cliente envió el texto: " + texto);
        System.out.println("El cliente envió un objeto con: " + objeto.info);
        return "Servidor: ¡Datos recibidos con éxito!";
    }

    @Override
    public Datos recibirDelServidor() throws RemoteException {
        System.out.println("Enviando un objeto desde el servidor...");
        return new Datos("Soy un objeto creado en el Servidor");
    }
}
```

```java
public class ServidorRMI {
    public static void main(String[] args) {
        try {
            // 1. Crear la instancia del objeto remoto
            HolaRMI objetoRemoto = new HolaRMI();

            // 2. Iniciar el registro RMI en el puerto 1099
            Registry registro = LocateRegistry.createRegistry(1099);

            // 3. Registrar el objeto con un nombre ("ObjetoHola")
            registro.rebind("ObjetoHola", objetoRemoto);

            System.out.println("Servidor RMI listo y esperando conexiones...");
        } catch (Exception e) {
            System.err.println("Error en el servidor: " + e.toString());
        }
    }
}
```

```java
public class ClienteRMI {
    public static void main(String[] args) {
        try {
            // 1. Obtener el registro RMI del servidor (localhost o IP)
            Registry registro = LocateRegistry.getRegistry("localhost", 1099);

            // 2. Buscar el objeto por su nombre
            IHolaRMI stub = (IHolaRMI) registro.lookup("ObjetoHola");

            // 3. ENVIAR AL SERVIDOR: Texto y Objeto
            Datos miObjeto = new Datos("Mensaje dentro de un objeto");
            String respuesta = stub.enviarAlServidor("Hola desde el Cliente", miObjeto);
            System.out.println("Respuesta del servidor: " + respuesta);

            // 4. RECIBIR DEL SERVIDOR: Obtener un objeto
            Datos objetoRecibido = stub.recibirDelServidor();
            System.out.println("Objeto recibido del servidor: " + objetoRecibido.info);

        } catch (Exception e) {
            System.err.println("Error en el cliente: " + e.toString());
        }
    }
}
```